CSC 106 Academic Portfolio & Planner

This project report explains the structure, purpose, and interactivity of the academic portfolio site developed for CSC 106. The portfolio is a multi-page static website that presents the student's identity, academic path, sample projects, and contact information. It was built using Cursor IDE, which supported rapid drafting, iterating, and debugging of the HTML, CSS, and JavaScript across the pages.

1. Overview of the project

The site includes a Home page, About page, Projects page, Planner page, and Contact page. It uses shared navigation and footer sections across the site, plus a common stylesheet for design consistency. The project uses semantic HTML for structure, CSS Grid and Flexbox for layout, and vanilla JavaScript for the interactive features.

The main files are index.html, about.html, projects.html, planner.html, contact.html, styles.css, and script.js. The build is intentionally lightweight and easy to understand, which makes it a strong teaching example for front-end web development fundamentals.

2. Home page

The Home page is the landing page and introduction to the student portfolio. It contains a hero section with a profile image, the student's name, role description, and a short introduction to the site. This gives the portfolio a strong first impression and establishes its purpose immediately.

```
<section class="hero" aria-labelledby="student-name">
  <img class="hero-photo" src="images/profile.jpg" alt="Portrait of Abiodun Adekunle" />
  <div>
    <p class="kicker">Welcome</p>
    <h1 id="student-name">Abiodun Adekunle</h1>
    <p class="hero-role">Software Engineering Student · CSC 106 Term Project</p>
  </div>
</section>
```

This section uses CSS classes such as hero, hero-photo, and hero-role to shape the appearance and spacing. The design keeps the student profile front and center, reinforcing the academic and personal identity of the site. The page also includes a short course introduction and a video player, which helps present the student's work in a more engaging format.

3. About page

The About page explains the student's educational background, technical skills, and interests. It includes an academic table showing programme information, key focus areas, and programme periods. It also describes career goals and personal interests, giving the visitor a fuller picture of the student beyond the portfolio itself.

```
<table class="edu-table">
  <thead>
    <tr>
      <th scope="col">Institution</th>
      <th scope="col">Programme</th>
      <th scope="col">Focus</th>
      <th scope="col">Years</th>
    </tr>
```

```
  </thead>
</table>
```

This layout makes academic history readable and professional. The skills list highlights abilities gained through coursework, including HTML, CSS, JavaScript, responsive design, and form validation. The hobbies section humanizes the page and shows that the student has broader interests beyond coding.

4. Projects page

The Projects page presents work in a card-based grid. Each project card includes an image, a project title, a summary, technology tags, and links to a live demo or repository. This format is ideal for a portfolio because it communicates each project quickly and consistently.

```
<article class="project-card">
  <img src="images/project1.png" alt="StudyFlow dashboard preview" />
  <div class="project-body">
    <h2>StudyFlow Dashboard</h2>
    <p>A course overview board that groups weekly lectures, lab hours, and assignment checkpoints.</p>
    <div class="tags">
      <span class="tag">HTML</span>
      <span class="tag">CSS Grid</span>
      <span class="tag">JavaScript</span>
    </div>
  </div>
</article>
```

The project cards are responsive and adapt to smaller screens, which makes the page usable on mobile devices. This is especially important for student portfolios because recruiters and visitors often browse from phones.

5. Contact page

The Contact page contains a message form and an aside that lists other ways to reach the student. The form collects the user's name, email, phone number, and message. This makes it easy for potential collaborators or instructors to get in touch.

```
<form id="contact-form" novalidate>
  <input class="control" id="contact-name" type="text" />
  <input class="control" id="contact-email" type="email" />
  <input class="control" id="contact-phone" type="tel" />
  <textarea class="control" id="contact-message"></textarea>
  <button class="btn btn-primary" type="submit">Send message</button>
</form>
```

The page intentionally reminds the user that the message is checked in the browser before acceptance. This demonstrates front-end validation and improves the user experience by preventing incomplete or malformed submission.

6. Interactivity added and how it works

Several interactive behaviors were added to make the site feel modern and functional. The JavaScript file

CSC 106 Academic Portfolio & Planner

initializes the following features when the document is ready: the mobile menu, task planner, and contact-form validation. This setup prevents race conditions and ensures the elements exist before behavior is attached.

```
document.addEventListener("DOMContentLoaded", function () {
  initMobileNav();
  initPlanner();
  initContactForm();
});
```

This pattern is standard in vanilla JavaScript because it prevents errors that might happen if the script runs before the HTML is loaded. Once the DOM is ready, the functions attach event listeners and update the page dynamically.

Mobile menu

```
function initMobileNav() {
  let toggle = document.querySelector(".nav-toggle");
  let nav = document.querySelector(".site-nav");

  toggle.addEventListener("click", function () {
    let isOpen = nav.classList.toggle("open");
    toggle.setAttribute("aria-expanded", isOpen ? "true" : "false");
  });
}
```

The mobile menu hides the navigation on smaller screens and reveals it when the user taps the hamburger button. The CSS class .site-nav.open controls the visibility, while aria-expanded keeps the button state meaningful for accessibility tools and assistive technology.

Academic planner

```
let STORAGE_KEY = "cos106-planner-tasks";

function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

function loadTasks() {
  let raw = localStorage.getItem(STORAGE_KEY);
  return raw ? JSON.parse(raw) : [];
}

function toggleTask(id) {
  tasks = tasks.map(function (task) {
    if (task.id === id) {
      return Object.assign({}, task, { completed: !task.completed });
    }
    return task;
  });
  saveTasks();
```

```
  renderTasks();
}
```

The planner allows students to keep track of assignments or study tasks. Each task includes a title, course, due date, priority, and completion status. Because the app stores items in localStorage, the list persists during browser refreshes and can act as a simple personal study dashboard.

Form validation

```
function validateContactForm(form) {
  let name = document.getElementById("contact-name");
  let email = document.getElementById("contact-email");
  let phone = document.getElementById("contact-phone");

  if (!name.value.trim()) {
    errors.push("Name cannot be empty.");
    name.classList.add("error");
  }

  if (!email.value.trim()) {
    errors.push("Email address cannot be empty.");
  } else if (!isValidEmail(email.value.trim())) {
    errors.push("Please enter a valid email address.");
  }
}
```

The validation logic checks for missing fields, invalid email addresses, and phone numbers that contain letters or symbols. It also updates the interface with visual feedback to show whether the user's input passed or failed validation. This is an important pattern for modern web forms because it reduces frustration and improves accuracy.

7. Styling and responsiveness

The stylesheet uses CSS variables, grid layouts, and media queries to keep the site cohesive and responsive. The colour palette, typography, and card treatments are consistent across all pages, while responsive rules ensure the layout collapses gracefully on smaller screens.

```
:root {
  --color-ink: #12232e;
  --color-navy: #1b3a4b;
  --color-teal: #0f766e;
  --color-gold: #c4a35a;
  --color-cream: #f8f4ef;
}

.hero {
  display: grid;
  grid-template-columns: 280px 1fr;
  gap: 2.5rem;
}
```

```
@media (max-width: 900px) {
  .hero, .bio-grid, .project-grid {
    grid-template-columns: 1fr;
  }
}
```

This CSS design keeps the site readable and professional without relying on external frameworks. It highlights the importance of semantic HTML pairing with thoughtful CSS for creating a polished front-end experience.

8. Cursor IDE and workflow

This project was built using Cursor IDE, an AI-assisted coding environment. Cursor helped with generating initial markup, refining the design, troubleshooting JavaScript errors, and iterating on small improvements across the site. The workflow was efficient because changes could be tested quickly and adjusted without losing momentum.

In a classroom assignment, using Cursor IDE demonstrated how AI-assisted development can support learning and iteration. It is especially helpful when designing a small portfolio project with several different page layouts and interactive features.

9. Conclusion

This project is a comprehensive front-end portfolio that shows both design skill and coding understanding. The Home, About, Projects, and Contact sections each serve a distinct purpose, while the interactive JavaScript features add usability and real-world learning value. The result is a polished academic portfolio that reflects strong web development fundamentals and thoughtful user experience design.

Overall, the project demonstrates how HTML structure, CSS styling, and JavaScript interactivity can work together to create a functional and professional portfolio website. It is accessible, responsive, and practical-qualities that are essential in modern front-end development.